Introdução à Computação
tipos abstratos, pilhas

Alexandre Rademaker

tipos abstratos de dados e estruturas de dados I

Um tipo abstrato de dados (ADT) é um modelo para tipos de dados definido por seu comportamento do ponto de vista de um usuário dos dados, especificamente em termos de valores possíveis, operações possíveis em dados de esse tipo e o comportamento dessas operações.

Uma estrutura de dados é um formato especializado para organizar, processar, recuperar e armazenar dados. Existem vários tipos (básicos e avançados) de estruturas de dados, geralmente projetadas para finalidades específicas como garantir uma dada complexidade para alguma operação.

tipos abstratos de dados e estruturas de dados II

O tipo `int` de C é um tipo de dados que implementa o tipo de dados abstrato inteiro.

Inteiros podem ser representados como números binários, na maioria das vezes como **complemento de dois**, mas podem ser **decimais codificados em binário** ou em **complemento de um**.

Usamos inteiros e as operações sobre inteiros sem nos preocupar com a implementação binária dos inteiros e como as operações são implementadas em termos da codificação binária.

tipos abstratos de dados e estruturas de dados III

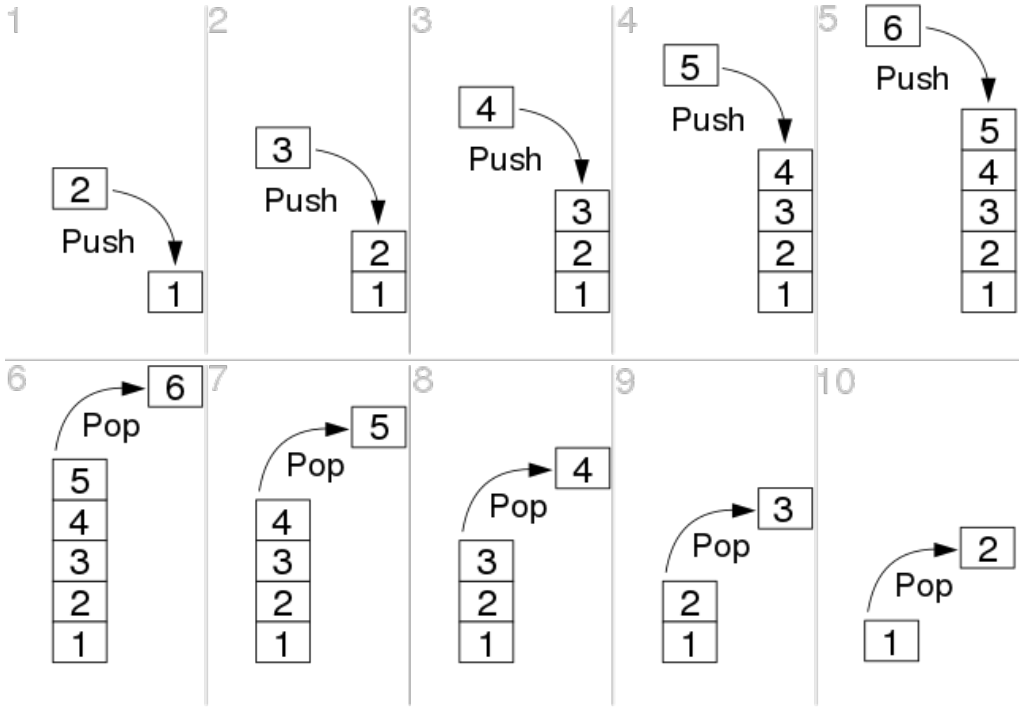
O tipo abstrato de dados lista (ou sequência) representa uma coleção de valores ordenada.

Sequências podem ser implementadas como **arrays** ou **listas encadeadas**.

As operações mais comuns são: incluir um elemento, acessar o nésimo elemento, remover um elemento, localizar um elemento, concatenar duas sequências, ordenar, etc.

pilhas I

Em uma pilha (stack), a manipulação dos elementos é realizada em apenas uma das extremidades, chamada de topo, em oposição a outra extremidade, chamada de base.



pilhas II

Pilha um tipo abstrato de dados do tipo "Last-In, First Out" (LIFO), ou seja "o último que entra é o primeiro que sai".

Pilhas podem ser implementadas em estruturas de dados concretas como arrays e listas encadeadas.

pilhas III

As operações básicas de uma pilha são:

push	insere um novo elemento no topo
pop	remove o elemento do topo
peek	retorna o elemento no topo sem remover da pilha
isEmpty	testa se a pilha está vazia.
size	retorna o tamanho da pilha, quantos elementos ela contém.

Todas podem ter complexidade $O(1)$ quando pilhas são implementadas usando arrays ou listas encadeadas.

pilhas IV

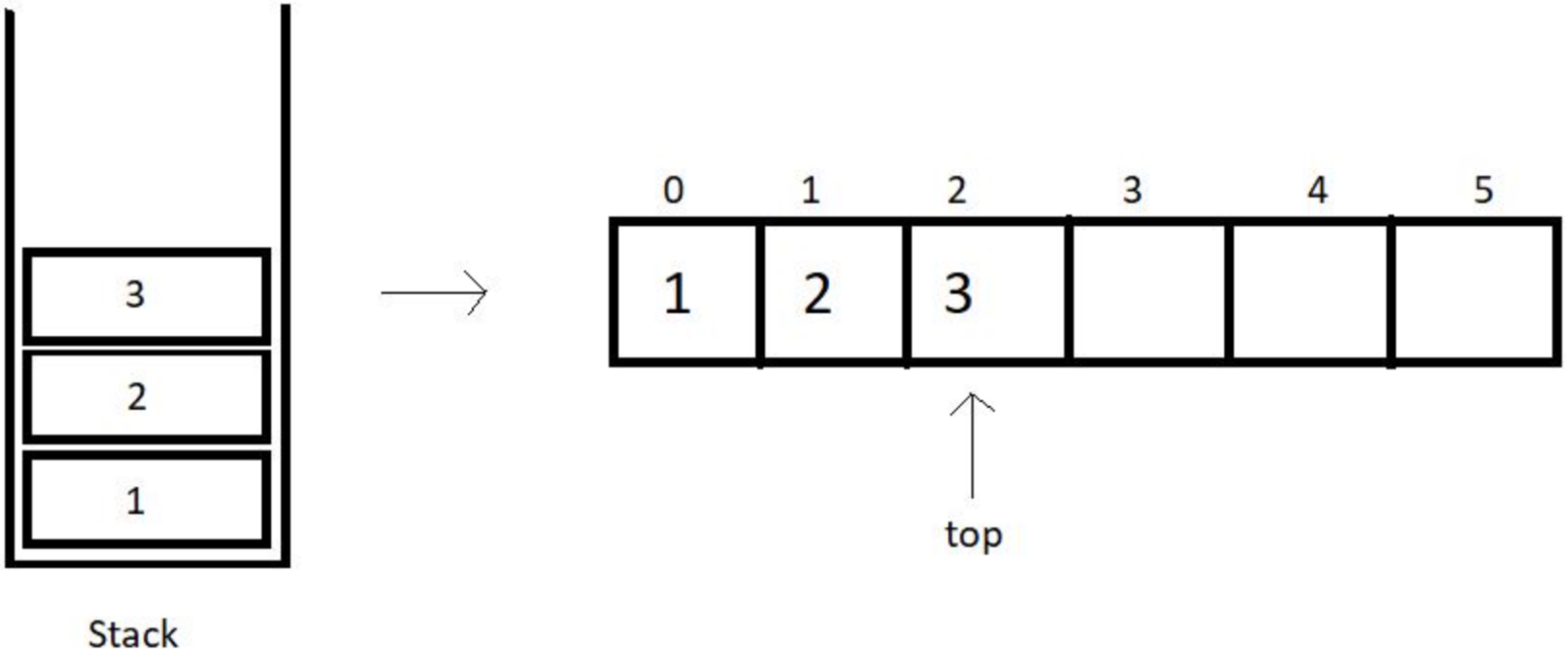
Pilhas podem ser usadas para reverter uma sequência (lista encadeada ou array) e ordenar uma sequência.

Verificar se uma expressão aritmética tem os parênteses balanceados. Por exemplo, a expressão $(2 + (8/y))$ e a expressão $(2 + (8/y)$.

Implementar um mecanismo de 'desfazer' operações.

Pilhas são ainda usadas para implementar buscas em grafos, conversão de expressões aritméticas em notação infixada $(2 + 3)$ para pré-fixada $(+23)$ ou pós-fixada $(23+)$.

pilhas usando array I



pilhas usando array II

```
1 // maximum size of the stack
2 #define MAX_SIZE 100
3
4 typedef struct stack {
5   int stack[MAX_SIZE];
6   int top;
7 } t_stack;
```

Implementando com array! Desvantagem, alocação estática com limite pré-definido de elementos. Vantagem, simplicidade.

pilhas usando array III

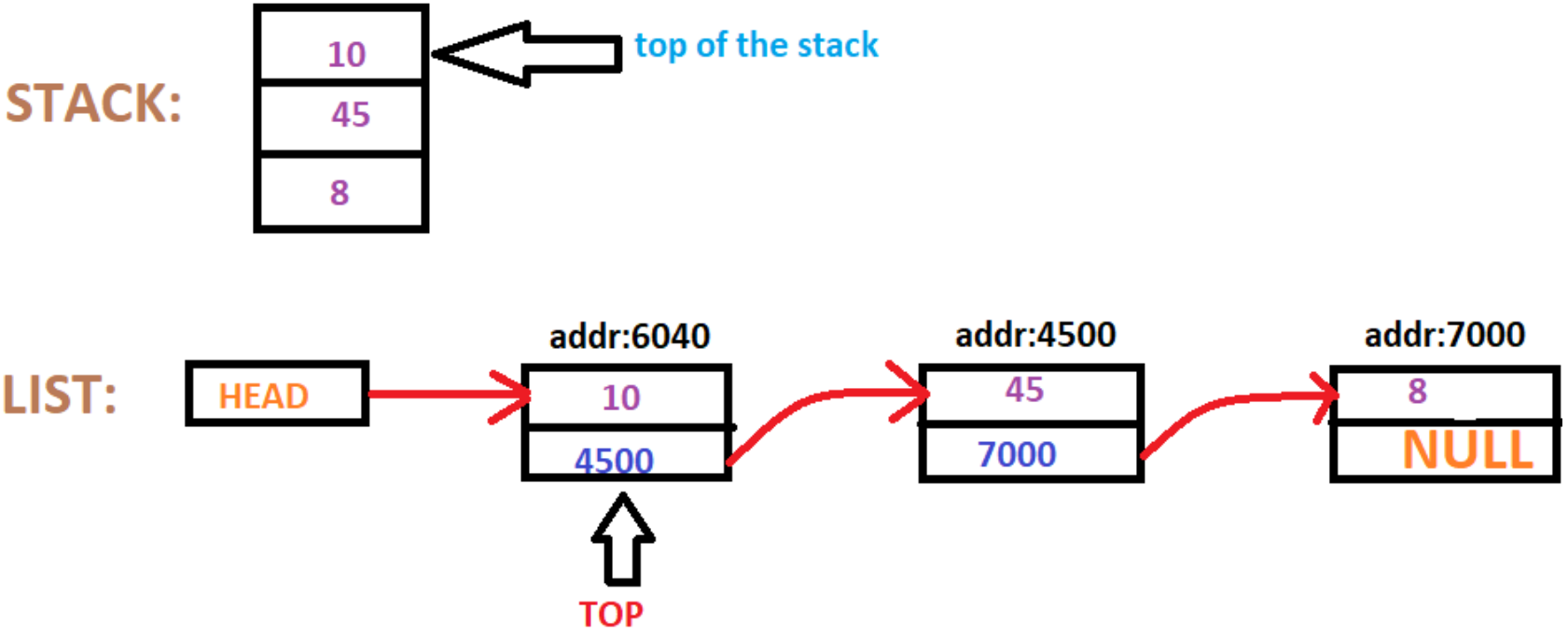
src/stack-array.c

```c
t_stack* make_stack(){
  t_stack *t = malloc(sizeof(t_stack));
  t->top = -1;
  return t;
}

void push(t_stack *s, int data) {
  assert(s->top < MAX_SIZE - 1);
  s->stack[++s->top] = data;
}

int pop(t_stack *s) {
  assert(s->top > -1);
  return s->stack[s->top--];
}
```

pilhas usando listas encadeadas I



pilhas usando listas encadeadas II

```
1 typedef struct node {
2   int number;
3   struct node *next;
4 } t_node;
```

Mesma `struct` declarada anteriormente para listas encadeadas.
Nenhuma inicialização é necessária.

pilhas usando listas encadeadas III

src/stack-list.c

```
int pop(t_node **p) {
  assert(*p != NULL);
  int r = (*p)->number;
  t_node *t = *p; *p = (*p)->next; free(t);
  return r;
}

void push(t_node **p, int e) {
  t_node* n = malloc(sizeof(t_node));
  assert(n != NULL); n->number = e; n->next = *p;
  *p = n;
}

int peek(t_node *p) {
  assert(p != NULL); return p->number;
}
```

pilhas usando listas encadeadas IV

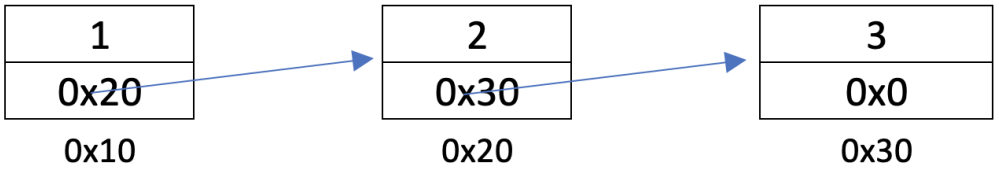
```
1 int main(int argc, char *argv[]) {
2   t_node *s = NULL;
3   push(&s, 1);
4   push(&s, 2);
5   push(&s, 3);
6
7   printf("popped %d \n", pop(&s));
8 }
```

push e pop alteram a pilha, por isso passamos o endereço de s (endereço da variável que é um endereço de um t_node) não o valor de s (endereço de um t_node).

push e pop alteram o valor de s, declarada na main.

pilhas usando listas encadeadas V

endereço	valor
...	...
0x10	1
0x14	0x20
...	...
0x20	2
0x24	0x30
...	...
0x30	3
0x34	NULL
...	
0x60	0x10
...	...
0x80	?




```
main:
        t_node *s;
        pop(&s);

pop(t_node **p):
        *p is 0x10
        *p->next is 0x20
        *p = (*p)->next;

pop(t_node *p):
        p = p->next;
```

p

0x60

0x80

p

0x10

0x80